
ENGENHARIA DE COMPUTAÇÃO

Disciplina: Engenharia de Software

KA 06 — Operações de Engenharia de Software

(Software Engineering Operations)

Plano de Implantação, Operação e Pipeline CI/CD

Instituição de Ensino	UNOESC
Professor(a) Responsável	Odemir Moreira da Mata Jr.
Disciplina	Engenharia de Software
Turma / Semestre	JBA620-6
Integrantes do Grupo	Ana Clara Viganó Prestes de Oliveira, Davi Antônio Basotto Minati, Elian Baldasso Pereira Duarte, Jasmine Masson Alves, Lucas Eduardo Heydt, Vitória Aparecida Vendausen
Data de Entrega	24/06/2026

Histórico de Versões

Versão	Data	Autor(es)	Descrição	Revisado por
v1.0	29/05/2026	Jasmine Alves	Definição inicial da estratégia operacional.	Lucas Heydt
v2.0	24/06/2026	Elian Duarte	Consolidação final do plano de implantação.	Vitória Vendausen

1. Estratégia de Implantação

1.1 Estratégia Escolhida e Justificativa

A estratégia adotada para o SisConsult é a **Implantação em Nuvem Containerizada** utilizando clusters Kubernetes gerenciados (como AWS EKS ou Google GKE). Os microsserviços da aplicação serão empacotados em imagens Docker isoladas.

Justificativa: A utilização de containers facilita a implantação e a manutenção da aplicação, pois cada serviço possui suas próprias dependências isoladas. Além disso, a infraestrutura pode ser ampliada conforme o aumento da demanda, contribuindo para os requisitos de disponibilidade e desempenho do sistema.

1.2 Definição dos Ambientes

O ciclo de vida do software operará com três ambientes isolados logicamente via Redes Virtuais Privadas (VPCs) e Namespaces dedicados:

Desenvolvimento (Dev): Ambiente dinâmico voltado para testes funcionais rápidos da equipe de engenharia. Possui recursos computacionais mínimos e ferramentas de depuração ativas.

Homologação/Staging (Homolog): Réplica idêntica à infraestrutura de produção em menor escala. Destina-se à execução de testes de carga, validação de segurança e homologação de novas features pelo Product Owner (PO).

Produção (Prod): Ambiente utilizado pelos usuários finais. Possui mecanismos de alta disponibilidade, monitoramento contínuo e controle de acesso para garantir estabilidade e segurança.

1.3 Procedimento de Promoção entre Ambientes

O fluxo de promoção de código adota o modelo GitOps baseado em ramificações (Trunk-Based Development):

Dev: Acionado automaticamente via Push para ramificações de feature branches.

Homolog: A promoção ocorre através da abertura e aceitação de um Pull Request (PR) direcionado à branch main. O merge dispara automaticamente a esteira de CI/CD para atualização do ambiente de Staging.

Prod: A implantação em produção ocorre após a aprovação do responsável técnico e a validação dos testes realizados em homologação.

| 1.4 Estratégia de Rollback

Será utilizada a estratégia de **Canary Deployment** combinada com **Blue/Green** para serviços de banco de dados.

Ao promover uma nova versão para produção, o roteador de tráfego direciona inicialmente apenas 10% das requisições para o novo container (*Canary*). O pipeline monitora ativamente a taxa de erros HTTP 5xx e latência nos primeiros 5 minutos. Caso qualquer métrica ultrapasse os limites aceitáveis do SLO, o tráfego é reorientado 100% de volta para a versão estável de forma automatizada, sem interrupção para o usuário. Para alterações estruturais de banco de dados, scripts de migração reversa (*Rollback Migrations*) são testados previamente em homologação.

| 1.5 Política de Dados de Teste (LGPD)

Em conformidade estrita com os artigos 6º (Minimização) e 13º da LGPD, é terminantemente proibida a utilização de dados reais de pacientes em ambientes de Desenvolvimento e Homologação.

Os dados destes ambientes serão gerados via scripts de sementeira automatizados (Seeding Scripts) utilizando bibliotecas de dados sintéticos (como a biblioteca Faker adaptada para padrões brasileiros). O script gerará CPFs válidos matematicamente, nomes, telefones e históricos médicos totalmente fictícios. Para evitar exposição de informações sensíveis, apenas dados fictícios serão utilizados nos ambientes de desenvolvimento e homologação.

2. Plano de Operação e Monitoramento

| 2.1 Métricas Operacionais a Monitorar

O monitoramento será baseado em indicadores relacionados ao desempenho e à estabilidade da aplicação, como latência, tráfego, taxa de erros e utilização de recursos.

Latência: Tempo gasto para processar as requisições HTTP e queries do banco de dados (segmentado em percentis p50, p95 e p99).

Tráfego: Demanda medida em requisições por segundo (RPS) que entram no balanceador de carga.

Erros: Taxa de requisições que falham (erros HTTP 5xx ou falhas de conexão interna).

Saturação: Grau de consumo dos recursos computacionais da infraestrutura (Uso de CPU, memória RAM, IOPS de disco e esgotamento de conexões no pool do banco de dados).fas de engenharia para evitar a reincidência do problema.

| 2.2 Definição de SLA

O SisConsult deverá manter um Acordo de Nível de Serviço (SLA) de disponibilidade mínima mensal de 99,5% para o ambiente de Produção, excluindo períodos previamente comunicados de manutenção programada.

Esse SLA corresponde a um tempo máximo de indisponibilidade não planejada de aproximadamente 3 horas e 39 minutos por mês. O monitoramento será realizado continuamente por meio das métricas de disponibilidade (SLI), calculadas a partir da fórmula:

$$\text{Disponibilidade (\%)} = ((\text{Tempo Total} - \text{Tempo de Inatividade}) / \text{Tempo Total}) \times 100$$

A garantia desse nível de serviço está diretamente relacionada aos Requisitos Não Funcionais definidos no KA 01, especialmente os requisitos de disponibilidade, confiabilidade e continuidade operacional do sistema. Para assegurar o cumprimento do SLA serão utilizados:

- Implantação em infraestrutura de alta disponibilidade (Multi-AZ);
- Balanceamento de carga entre instâncias da aplicação;
- Escalonamento automático de recursos (Horizontal Pod Autoscaler);
- Monitoramento contínuo de serviços e infraestrutura;
- Estratégias automatizadas de rollback por meio de Canary Deployment;
- Procedimentos formais de resposta e mitigação de incidentes.

O não atendimento ao SLA durante o período de medição gera a abertura obrigatória de análise de causa raiz (Root Cause Analysis – RCA) e revisão dos mecanismos de prevenção e recuperação de falhas

| 2.3 Procedimento de Gestão de Incidentes

O ciclo de vida de contenção de falhas seguirá o fluxo padronizado:

Identificação: Sistemas de monitoramento (Prometheus/Grafana) disparam alertas automatizados via canais de Webhook para a plataforma de chamados (PagerDuty) ao detectarem anomalias nos SLOs.

Escalação: Caso o incidente não receba o aceite do engenheiro de plantão (SRE) em até 10 minutos, o PagerDuty escala automaticamente o chamado para o Engenheiro DevOps L2 e, em seguida, para o Tech Lead.

Resolução: Aplicação imediata de contramedidas de mitigação (ex: ampliação forçada de nós, isolamento de réplicas de leitura corrompidas do banco de dados ou execução de Rollback).

Post-Mortem: Em até 48 horas após a resolução, o time se reúne para confeccionar um relatório de Post-Mortem Sem Culpa (Blameless Post-Mortem), analisando a causa raiz e gerando tarefas de engenharia para evitar a reincidência do problema.

| 2.4 Gerenciamento de Segredos e Credenciais por Ambiente

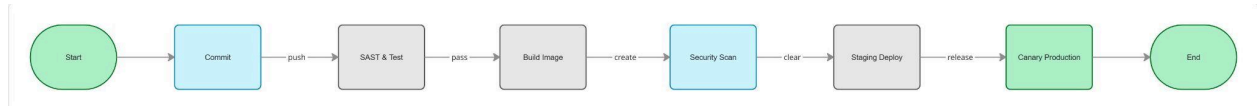
Nenhuma credencial, token de API, chave criptográfica ou string de conexão com bancos de dados poderá ser armazenada diretamente no código-fonte ou no repositório Git.

O gerenciamento de segredos será realizado por meio de cofres de chaves gerenciados (como AWS Secrets Manager ou HashiCorp Vault). Os segredos são injetados dinamicamente

nos containers em tempo de execução (Runtime) como variáveis de ambiente injetadas de forma segura pelo orquestrador. Cada ambiente possui seu próprio cofre isolado e as chaves de acesso são rotacionadas automaticamente a cada 90 dias, limitando o privilégio de visualização apenas ao pipeline autenticado.

3. Pipeline de CI/CD

3.1 Diagrama do Pipeline (inserir imagem)



3.2 Descrição de Cada Etapa

Cada etapa descrita abaixo atua como uma barreira de qualidade obrigatória. A falha em qualquer uma delas interrompe imediatamente o pipeline e notifica a equipe desenvolvedora:

1. Linting e Análise Estática (SAST): Varre o código-fonte em busca de más práticas de digitação, checagem de regras de arquitetura e detecção de vulnerabilidades conhecidas no código usando ferramentas como SonarQube.
2. Testes Unitários e Integração: Executa a suíte de testes de código automatizados garantindo que nenhuma alteração quebre regras de negócio pré-existent.
3. Análise de Vulnerabilidade de Dependências (SCA): Analisa bibliotecas de terceiros integradas ao projeto (package.json, pom.xml, etc.) à procura de falhas de segurança listadas no banco de dados do CVE/NVD.
4. Compilação e Build da Imagem: Cria a imagem Docker imutável do microserviço utilizando builds multiestágio para otimizar o tamanho final do arquivo.
5. Varredura da Imagem Container (Vulnerability Scan): Inspecciona as camadas do sistema operacional base da imagem Docker (utilizando ferramentas como Trivy) à procura de falhas antes de exportá-la.
6. Publicação no Registro (Push): Armazena o container validado e tagueado com o hash do commit em um registro privado e seguro (como AWS ECR).
7. Implantação Automatizada em Homologação: Atualiza os manifestos do Kubernetes via ferramentas GitOps (como ArgoCD), aplicando a nova imagem no namespace de Homologação.
8. Testes de Fumaça (Smoke Tests) e Promoção: Executa um conjunto de requisições de teste fim-a-fim automatizadas para confirmar a saúde da aplicação. Após a validação e liberação manual do gestor, a imagem avança para o cluster de Produção no modelo Canary.

Tabelas e Formulários

Definição dos Ambientes

Ambiente	Finalidade	Infraestrutura	Dados Utilizados	Acesso Permitido
Desenvolvimento	Testes funcionais rápidos e codificação da equipe.	Cluster local de minikube ou Pods isolados em Namespace Dev de baixa especificação.	Dados fictícios gerados via scripts automatizados (Faker).	Desenvolvedores e Engenheiros de Software do projeto.
Homologação/Staging	Validação de QA, testes de estresse e homologação pelo PO.	Réplica do ambiente produtivo em escala reduzida (25% da capacidade de nós).	Dados fictícios complexos ou dados mascarados/anonimizados.	Equipe de QA, Engenheiros DevOps e Product Owner.
Produção	Atendimento em tempo real aos usuários do SisConsult.	Cluster de Alta Disponibilidade (Multi-AZ) com escalonamento HPA ativo.	Dados reais protegidos por criptografia em repouso e trânsito (LGPD).	Usuários finais (médicos/pacientes) e engenheiros SRE em regime de auditoria.

| Etapas do Pipeline CI/CD

#	Etapa	Descrição	Ferramenta	Critério de Sucesso
1	Code Linting & SAST	Análise estática do estilo e checagem de segurança do código-fonte.	ESLint / SonarQube	Zero bugs críticos e cobertura mínima de qualidade aceita.
2	Unit Testing	Execução dos testes automatizados de menor granularidade.	Jest / JUnit	100% dos testes com status aprovado e cobertura de código $\geq 85\%$.
3	SCA Scan	Varredura de segurança em bibliotecas externas importadas.	OWASP Dependency-Check	Bloqueio imediato caso existam vulnerabilidades de nível Alto ou Crítico.
4	Container Build	Empacotamento do microsserviço em uma imagem Docker.	Docker CLI / BuildKit	Geração da imagem compilada com sucesso e sem erros de pipeline.
5	Image Security Scan	Varredura interna nas camadas de SO da imagem gerada.	Trivy / Aqua Security	Ausência de vulnerabilidades críticas conhecidas no ecossistema do

				container.
6	Artifact Storage	Envio e armazenamento da imagem Docker em repositório seguro.	AWS ECR / Docker Hub	Upload concluído com sucesso com tag única associada ao Commit ID.
7	Staging Deployment	Atualização automática do ambiente de Homologação.	ArgoCD / kubectl	Sincronização bem-sucedida do cluster e pods em estado Running.
8	Smoke & Gate Release	Testes de fumaça seguidos por portão de aprovação para produção.	Newman (Postman) / Manual	Testes funcionais em 100% e assinatura digital do gestor liberando a tag.

| Métricas Operacionais (SLI/SLO)

Indicador (SLI)	Objetivo (SLO)	Método de Medição	Frequência	Responsável
Disponibilidade	≥99,5%	$SLI = \frac{\text{Tempo Total} - \text{Tempo de Inatividade}}{\text{Tempo Total}} \times 100$	Mensal	SRE / DevOps
Tempo de resposta (p95)	<2s	Percentil 95 do tempo de ida e volta (RTT) medido no API Gateway.	Contínuo (Alertas em 5 min)	Engenheiro de Infraestrutura
Taxa de erros HTTP 5xx	<0,1%	$SLI = \frac{\text{Total de Requisições} - \text{ColetadasRequisições com Erro 5xx}}{\text{Total de Requisições}} \times 100$	Tempo Real (Média móvel)	Equipe de Operações

| Observações e Referências Consultadas

SWEBOK v4.0 (Guide to the Software Engineering Body of Knowledge): IEEE Computer Society, 2024. Consulta ao Capítulo 7 (*Software Engineering Management*) focado em gerência de configuração e processos de liberação (*Release*).

Lei Geral de Proteção de Dados Pessoais (LGPD): Lei nº 13.709/2018, Governo Federal do Brasil. Princípios de segurança e limitação de acesso aplicados às políticas de dados sintéticos e restrição de ambientes de teste.

Google Site Reliability Engineering (SRE Handbook): B. Beyer, C. Jones, J. Petoff e N. R. Murphy. Princípios de monitoramento (Four Golden Signals), gerenciamento de incidentes de software e modelagem de SLI/SLO/SLA.

ISO/IEC/IEEE 29119-4: *Software and systems engineering — Software testing*. Normativas de processos de teste aplicadas à validação contínua da esteira de integração do pipeline CI/CD.